

layerops

Assess and remediate container image layers. Answers how many vulnerabilities can actually be fixed across one image or a hundred — then, on request, fixes the ones in the base layer.

CONTENTS

- | | |
|------------------------------------|---|
| 01 Name | 08 Patching |
| 02 Synopsis | 09 CSV format |
| 03 Description | 10 HTML report |
| 04 Installation | 11 Examples & recipes |
| 05 Options | 12 Exit status |
| 06 Output | 13 Notes |
| 07 Sum vs distinct | 14 Troubleshooting |

01 NAME

layerops — summarise container image vulnerabilities as total vs fixable, split by package class.

02 SYNOPSIS

SYNOPSIS

```
layerops --image <repo:tag> [--image <repo:tag> ...] [options]
layerops --list <file> [options]
layerops --from-json <file> [options]
```

03 DESCRIPTION

Trivy's default table prints every finding line by line. When you are reporting on an estate of images, what you need instead is the shape of the problem: how much is real work, and how much is noise you cannot act on. This tool runs the scan and collapses it into that shape.

Every finding is bucketed two ways.

By package class

os-pkgs

Distro packages — rpm, deb, apk. The base image layer. Fixed by rebasing onto a newer base image, not by changing your code.

lib-pkgs

Language packages — jar, npm, pip, Go modules. The application layer. Fixed by bumping dependencies or upgrading the app.

By fix availability

A finding is **FIXABLE** when the vulnerability database carries a fixed version for the installed package. It is **UNFIXED** when it does not — the vendor has triaged the CVE and shipped no patch, or marked it `will_not_fix`, `fix_deferred` or `end_of_life`. Unfixed findings still matter for risk acceptance, but no amount of patching clears them, so they do not belong in a remediation backlog.

WHY THIS SPLIT MATTERS

An image can report 758 vulnerabilities of which only 8 have no patch. Another can report 40 with 35 unfixed. The raw count ranks those two wrongly; the fixable count ranks them correctly.

Requirements

- `trivy` — required for scanning (`brew install trivy`). Not needed with `--from-json`.
- `jq` — required always (`brew install jq`).
- `column` — optional, used for alignment. Output degrades to tab-separated without it.

Trivy is always invoked with `--scanners vuln`, so secret scanning is skipped and scans run noticeably faster than a bare `trivy image`.

04 INSTALLATION

BASH

```
sudo mv layerops /usr/local/bin/  
sudo chmod +x /usr/local/bin/layerops  
  
# macOS only: clear the download quarantine flag  
xattr -d com.apple.quarantine /usr/local/bin/layerops  
  
# verify  
which layerops  
layerops --help
```

If `/usr/local/bin` does not exist, create it with `sudo mkdir -p /usr/local/bin`. Running the script from a directory not on your `PATH` requires the `./` prefix.

05 OPTIONS

Input — pick one

FLAG	ARGUMENT	DESCRIPTION
-i, --image	<ref>	Image to scan. Repeat the flag to pass several. A bare argument with no flag is also treated as an image.
-l, --list	<file>	File of image references, one per line. Blank lines skipped, # starts a comment. Combinable with --image .
-f, --from-json	<file>	Summarise an existing Trivy JSON report without scanning anything.

Output

FLAG	ARGUMENT	DEFAULT	DESCRIPTION
-c, --class	os-pkgs lib-pkgs all	all	Which classes to print. Filters display only — the scan still covers everything --pkg-types allows.
--csv	<file>	—	Write the full report as CSV — per-image breakdown, severity totals, and distinct CVE counts.
--json-dir	<dir>	—	Keep each image's raw Trivy JSON here instead of a temp dir deleted on exit.
-o, --save-json	<file>	—	Single-image mode only: keep that image's raw Trivy JSON at this path.

Scan behaviour

FLAG	ARGUMENT	DEFAULT	DESCRIPTION
-p, --pkg-types	<list>	os,library	Passed to Trivy as <code>--pkg-types</code> . Controls what is actually scanned.
-s, --severity	<list>	all	Limit to given severities, e.g. <code>CRITICAL,HIGH</code> . Applied at scan time.
-q, --quiet	—	off	Suppress Trivy's own progress output. The tool's <code>[n/N] scanning</code> lines still print to stderr.
--fail-on-fixable	—	off	Exit 1 if any fixable vulnerability is found across every image scanned. For CI gates.
-h, --help	—	—	Print usage and exit 0.
--patch	—	off	Patch fixable <code>os-pkgs</code> findings with copa. Writes a new local tag; the source image is never modified.
--patch-suffix	<s>	lo-delta	Tag suffix for patched images. <code>nginx:1.30.3</code> becomes <code>nginx:1.30.3-lo-delta</code> .
--verify	—	off	Re-scan each patched image and report before → after. Without it, copa's exit status is taken at face value.
--copa-addr	<addr>	auto	BuildKit address, passed to <code>copa -a</code> , e.g. <code>tcp://0.0.0.0:8888</code> .
--copa-timeout	<d>	copa default	Per-image patch timeout, e.g. <code>5m</code> .
--html	<file>	—	Write a standalone HTML report listing every CVE, tabbed by <code>os-pkgs</code> and <code>lib-pkgs</code> . Searchable, filterable, self-contained.
-qc, --copa-quiet	—	follows -q	Hide copa's BuildKit progress bars and apt chatter. If an image fails, its captured output is printed anyway, so a quiet run never hides a reason.
-Qc, --copa-verbose	—	—	Show copa's output even under <code>-q</code> .

06 OUTPUT

Single image

One image gets an identity block naming the detected distro and library package types, then one severity table per class with the type repeated in the heading.

```
LAYEROPS --IMAGE QUAY.IO/KEYCLOAK/KEYCLOAK:26.7.2
```

```
IMAGE: quay.io/keycloak/keycloak:26.7.2
OS:    redhat 9.8
LIB-PKGS: jar
```

=== os-pkgs (redhat 9.8) ===

SEVERITY	TOTAL	FIXABLE	UNFIXED
HIGH	3	2	1
MEDIUM	9	1	8
LOW	22	0	22
TOTAL	34	3	31

=== lib-pkgs (jar) ===

SEVERITY	TOTAL	FIXABLE	UNFIXED
HIGH	1	1	0
MEDIUM	2	2	0
TOTAL	3	3	0

Multiple images

Two or more images add a per-image table ahead of each master total, and a distinct-CVE line beneath it.

```
LAYEROPS --LIST IMAGES.TXT --CLASS OS-PKGS
```

=== os-pkgs - per image ===

IMAGE	TYPE	TOTAL	FIXABLE	UNFIXED
airbyte/server:2.1.0	debian 12.5	152	152	0
airbyte/worker:2.1.0	debian 12.5	152	152	0
airbyte/cron:2.1.0	debian 12.5	168	168	0
temporalio/auto-setup:1.27.2	debian 12.5	64	64	0
minio/minio:RELEASE.2023-...	redhat 8.9	70	62	8

=== os-pkgs - master total (5 images) ===

SEVERITY	TOTAL	FIXABLE	UNFIXED
CRITICAL	2	2	0
HIGH	458	458	0
MEDIUM	238	236	2
LOW	60	54	6
TOTAL	758	750	8

distinct CVEs: 158 distinct fixable: 152 distinct unfixed: 6

The numbers above illustrate the format; they are not a real scan.

Column meanings

COLUMN	MEANS
IMAGE	The reference as you supplied it, or the filename under <code>--from-json</code> .
TYPE	For <code>os-pkgs</code> , the detected distro and version — <code>debian 12.5</code> , <code>redhat 9.8</code> , <code>alpine 3.19</code> . For <code>lib-pkgs</code> , the package ecosystems present — <code>jar</code> , <code>gobinary</code> , <code>node-pkg</code> . A dash means Trivy detected none.
TOTAL	Every finding in that class.
FIXABLE	Findings carrying a fixed version. Your actual patch backlog.
UNFIXED	Findings with no fix available. Risk-acceptance material, not backlog.

Severity rows always sort CRITICAL, HIGH, MEDIUM, LOW, UNKNOWN, regardless of which are present. A severity with no findings is omitted rather than printed as a zero row.

07 SUM VS DISTINCT

The master `TOTAL` row sums occurrences. If twelve images share one base layer, a single CVE in that base is counted twelve times. That is the correct answer to "how many findings does our scanner report" and the wrong answer to "how many problems do we have".

The **distinct** line beneath each master total counts unique CVE IDs across the whole batch. For remediation planning — where fixing one base image clears the finding everywhere at once — distinct is the honest number.

RULE OF THUMB

Report **sum** to describe scanner output or per-image posture. Report **distinct** to size the work. A large gap between them means your estate shares base images — good news, because one rebase fixes many rows.

08 PATCHING

`--patch` turns the report into an action. For each image, the fixable `os-pkgs` findings are handed to Copacetic (`copa`), which rebuilds the package layer in place and writes the result as a new tag.

OS PACKAGES ONLY

`copa` patches distro packages — `apt`, `apk`, `yum`, `rpm`. It **cannot** patch `lib-pkgs`: a vulnerable jar, npm module, pip package or Go binary is compiled into the application and needs a dependency bump and a rebuild.

What actually runs

No second scan is needed. The Trivy JSON already in hand is filtered down to `os-pkgs` results carrying a `FixedVersion` — exactly what `trivy --pkg-types os --ignore-unfixed` would have produced — and passed to `copa` as its report, so patching is targeted rather than a blanket package update.

PER IMAGE, UNDER THE HOOD

```
copa patch -i nginx:1.30.3 -r copa-report.json -t 1.30.3-lo-delta
```

The patch table

LAYEROPS --LIST IMAGES.TXT --PATCH --VERIFY

```
=== patch - copa (os-pkgs only) ===
```

IMAGE	PATCHED TAG	OS BEFORE	FIXABLE	STATUS	OS AFTER
REMOVED					
docker.io/nginx:1.30.3	docker.io/nginx:1.30.3-lo-delta	404	167	PATCHED	237
167					
quay.io/thanos:v0.41.0	-	0	0	SKIPPED	-
-					
docker.io/drupal:latest	-	1718	366	FAILED	-
-					
patched: 1 failed: 1 skipped (nothing fixable): 1					

Status values

STATUS	MEANS
PATCHED	<code>copa</code> exited 0 and the new tag exists locally. With <code>--verify</code> , <code>OS AFTER</code> and <code>REMOVED</code> come from a real re-scan of that tag; without it they read <code>-</code> .
SKIPPED	Nothing for <code>copa</code> to do — zero fixable <code>os-pkgs</code> . Scratch, distroless and busybox images always land here, as do images whose OS findings are all unfixed.
FAILED	<code>copa</code> exited non-zero. The image is named in the run footer and the batch continues — one bad image never costs you the rest.

Safety

- The source image is never overwritten. Patching only ever creates `<repo>:<tag>-lo-delta`.

- Nothing is pushed. Results stay in the local image store until you push them yourself.
- Registry ports are handled correctly — `localhost:5000/app` is not mistaken for a tagged reference.
- Digest-pinned refs resolve to `<repo>:latest-lo-delta`, since a digest cannot carry a tag.

copa and a BuildKit endpoint are checked before scanning starts, so a missing dependency fails in a second rather than after a forty-image scan.

09 CSV FORMAT

`--csv` writes the full report in sections that mirror the terminal output — per-image breakdown, severity totals per class, distinct CVE counts, and, when `--patch` ran, the patch results.

REPORT.CSV — PER-IMAGE BREAKDOWN

```
image,os_type,os_total,os_fixable,os_unfixed,lib_type,lib_total,lib_fixable,lib_unfixed,grand_to
tal,grand_fixable,grand_unfixed
airbyte/server:2.1.0,amazon 2,152,152,0,jar,75,75,0,227,227,0
minio:RELEASE.2023-11-20,redhat 9.3,70,62,8,gobinary,246,243,3,316,305,11
```

REPORT.CSV — PATCH RESULTS (ONLY WITH --PATCH)

```
image,patched_tag,os_before,os_fixable,status,os_after,removed
docker.io/library/nginx:1.30.3,docker.io/library/nginx:1.30.3-lo-delta,404,167,PATCHED,237,167
quay.io/thanos/thanos:v0.41.0,-,0,0,SKIPPED,-,-
```

The CSV honours `--class`: filter the display and you filter the file.

10 HTML REPORT

`--html` writes a single self-contained file listing **every individual CVE**, not just the counts. It is the output to hand to someone who has to actually triage the findings.

BUILT TO BE SHARED

No CDN, no web fonts, no network calls — the report opens from a file path, on a laptop with no internet, and survives being emailed or dropped in a chat.

Headline figures

Fixable and Unfixed are counted against `os-pkgs` alone. `copa` patches OS packages and nothing else, so blending the classes would merge work a machine can do with work that needs a developer. Library findings get their own tile, and each class gets its own distinct-CVE count.

TILE	COUNTS
OS-pkg fixable	os-pkgs findings carrying a fixed version — the backlog <code>--patch</code> can clear.
OS-pkg unfixed	os-pkgs findings with no vendor patch. Risk acceptance, not backlog.
Library findings	Every lib-pkgs finding. A fix existing upstream still means a dependency bump and a rebuild.
Distinct CVEs · OS-pkg	Unique CVE IDs in os-pkgs across the estate.
Distinct CVEs · Libraries	Unique CVE IDs in lib-pkgs across the estate.

Each tile is a button, and the view it opens contains exactly as many rows as the number on the tile.

Tabs

TAB	CONTAINS
Overview	Headline figures, a severity distribution bar per class, and the per-image breakdown table.
Distro packages	Every <code>os-pkgs</code> finding. These are the ones <code>--patch</code> can act on.
Libraries	Every <code>lib-pkgs</code> finding — jar, gobinary, npm, pip. Separated because remediation is a different job.
Patch results	Only present when <code>--patch</code> ran. Per-image status with before and after counts.

Working through the findings

- **Three grouping modes.** *One row per distinct CVE* is the default and matches the distinct-CVE tiles exactly. *One row per CVE + package* is the remediation unit. *One row per image* gives raw occurrences.
- In distinct-CVE mode the fix column reads `partial` when some affected packages have a fix and others do not.
- **Search** matches CVE ID, package name, image reference and package type at once.
- **Severity chips** combine; **Fixable** and **Unfixed** narrow further. The severity bar re-renders against the current selection.

- **Every CVE ID links out** — NVD for `CVE-` identifiers, the GitHub advisory database for `GHSA-` ones.
- **Copy filtered as CSV** exports exactly what is on screen after filtering.

Scale

Findings are stored normalised — images and severities as indices, CVE titles held once rather than per occurrence — and the table paginates at 100 rows. Roughly 18,000 findings lands near 2 MB.

BASH

```
layerops --list images.txt --html report.html  
layerops --list images.txt --patch --verify --html report.html --csv report.csv
```

11 EXAMPLES & RECIPES

1 • Scan a single image

BASH

```
layerops --image quay.io/keycloak/keycloak:26.7.2
```

2 • Scan a whole estate, base layer only, into a spreadsheet

BASH

```
layerops --list images.txt --class os-pkgs --csv os-report.csv
```

3 • Build the image list from a live cluster

More reliable than transcribing tags out of a spreadsheet — these are the references actually running.

BASH

```
kubectl get pods -A -o jsonpath='{range .items[*]}{range .spec.containers[*]}{.image}{"\n"}{end}{end}' \
  | sort -u > images.txt

# include init containers too
kubectl get pods -A -o jsonpath='{range .items[*]}{range .spec.initContainers[*]}{.image}{"\n"}{end}{end}' \
  | sort -u >> images.txt

sort -u -o images.txt images.txt
```

4 • Build the image list from a compose file

BASH

```
docker compose config --images | sort -u > images.txt
```

5 • Only what would breach an SLA

BASH

```
layerops --list images.txt --severity CRITICAL,HIGH --quiet
```

6 • CI gate — fail the build on anything patchable

BASH • EXITS 1 WHEN FIXABLE FINDINGS EXIST

```
layerops --image "$IMAGE" --severity CRITICAL,HIGH --fail-on-fixable
```

7 • Keep raw reports, then re-cut without rescanning

Scanning a large estate is slow; re-cutting the same reports is instant.

BASH

```
# scan once, keep every report
layerops --list images.txt --json-dir ./reports

# later: re-read one report, no network, no pull
layerops --from-json ./reports/airbyte_server_2.1.0.json
```

8 • Compare two tags before an upgrade

Run each tag separately and read the FIXABLE column — if the new tag's fixable count is lower, the rebase is worth shipping.

BASH

```
layerops --image quay.io/keycloak/keycloak:26.7.1 --class os-pkgs
layerops --image quay.io/keycloak/keycloak:26.7.2 --class os-pkgs
```

9 • Weekly estate report, timestamped

BASH

```
DATE=$(date +%F)
layerops --list images.txt \
  --csv "vuln-$DATE.csv" \
  --json-dir "./reports/$DATE" \
  --quiet | tee "vuln-$DATE.txt"
```

10 • Ad-hoc set without writing a list file

BASH

```
layerops \
  -i airbyte/server:2.1.0 \
  -i airbyte/worker:2.1.0 \
  -i temporalio/auto-setup:1.27.2
```

11 • A list file with comments

```
IMAGES.TXT

# --- Airbyte platform ---
airbyte/server:2.1.0
airbyte/worker:2.1.0
airbyte/cron:2.1.0
airbyte/bootloader:2.1.0

# --- supporting services ---
temporalio/auto-setup:1.27.2
minio/minio:RELEASE.2023-11-20T22-40-07Z

# quarantined until the rebase lands
# legacy/app:0.9.4
```

12 EXIT STATUS

CODE	MEANING
0	Completed. Also the result when some images failed to scan but at least one succeeded, and when <code>--fail-on-fixable</code> is set and nothing fixable was found.
1	<code>--fail-on-fixable</code> was set and at least one fixable vulnerability exists.
2	Usage error, missing dependency, unreadable input, or every image failed to scan.

13 NOTES

The JSON holds more than the table shows

Trivy's human-readable table can display fewer findings than its JSON report contains for the same scan — vendor-suppressed statuses are filtered from the table view. This tool always reads the JSON, so its counts are the complete set. If its numbers look higher than the table you are used to, the JSON is right.

Relationship to --ignore-unfixed

Trivy's `--ignore-unfixed` drops unfixed findings at scan time. This tool never passes it, because it needs both sides to show the ratio. The FIXABLE column is what `--ignore-unfixed` would have left you with.

Failed pulls do not abort the run

An image that cannot be pulled is skipped with a warning and re-listed at the end of the run. One typo in a list of eighty does not cost you the batch. Check that footer before trusting a master total — a silently skipped image makes the estate look cleaner than it is.

Registry rate limits

Large batches against Docker Hub can hit anonymous pull limits. Authenticate first with `docker login`, or point Trivy at an internal mirror.

14 TROUBLESHOOTING

SYMPTOM	CAUSE AND FIX
command not found	The script is not on your <code>PATH</code> . Move it to <code>/usr/local/bin</code> , or run it as <code>./layerops</code> from its directory.
unidentified developer	macOS quarantined the downloaded file. Clear it: <code>xattr -d com.apple.quarantine <path></code> .
jq is required	<code>brew install jq</code> . Required even with <code>--from-json</code> .
Columns not aligned	<code>column</code> is missing; output falls back to tab-separated. Harmless, and still valid input for <code>cut</code> or a spreadsheet paste.
No vulnerabilities found	Genuinely clean, or <code>--severity</code> filtered everything out. Re-run without the filter to confirm.
SKIPPED (scan failed)	Bad reference, private registry needing <code>docker login</code> , or a rate limit. The failing refs are listed at the end of the run.

DATA SOURCE

This tool wraps Trivy. Vulnerability data and fix availability come from Trivy's databases — the tool aggregates, it does not assess.